

```
import discord
from discord.ext import commands
from discord import app_commands
import os
import json
import asyncio
from datetime import datetime, timedelta, timezone
from collections import defaultdict

OWNER_ID = 1408144132966322407

def is_owner(user: discord.Member) -> bool:
    return user.id == OWNER_ID

intents = discord.Intents.default()
intents.message_content = True
intents.members = True
intents.bans = True
intents.guilds = True
intents.moderation = True

bot = commands.Bot(command_prefix="!", intents=intents, help_command=None)
tree = bot.tree

WARNINGS_FILE = "warnings.json"
CONFIG_FILE = "config.json"

def load_warnings():
    if os.path.exists(WARNINGS_FILE):
        with open(WARNINGS_FILE, "r") as f:
            return json.load(f)
    return {}

def save_warnings(data):
    with open(WARNINGS_FILE, "w") as f:
        json.dump(data, f, indent=2)

def load_config():
    if os.path.exists(CONFIG_FILE):
        with open(CONFIG_FILE, "r") as f:
            return json.load(f)
    return {"alert_users": []}

def save_config(data):
```

```

        with open(CONFIG_FILE, "w") as f:
            json.dump(data, f, indent=2)

warnings_data = load_warnings()
config_data = load_config()

NUKE_WINDOW = 10
NUKE_THRESHOLD = 3
nuke_tracker = defaultdict(lambda: defaultdict(list))

# -----
# Liquid-Glass Embed Helper
# -----

def liquid_glass_embed(title: str, description: str = "", color: discord.Color =
    if color is None:
        color = discord.Color.from_rgb(140, 210, 255)

    divider = "`\n" + "..." * 34 + "\n`"
    structured_desc = (description + "\n" + divider) if description else divider

    embed = discord.Embed(
        title=f"🔮 {title}",
        description=structured_desc,
        color=color,
        timestamp=datetime.now(timezone.utc)
    )

    if fields:
        for name, value, inline in fields:
            embed.add_field(name=f"▶ {name}", value=value, inline=inline)

    embed.set_footer(text="◆ GermanyRP • System ◆")
    return embed

# -----
# Anti-Nuke
# -----

async def get_audit_executor(guild, action):
    try:
        async for entry in guild.audit_logs(limit=1, action=action):
            if (datetime.now(timezone.utc) - entry.created_at).total_seconds() <
                return entry.user
    except discord.Forbidden:
        pass

```

```

    return None

async def check_nuke(guild, user, action_label):
    if user is None or user.bot:
        return
    now = datetime.now(timezone.utc)
    cutoff = now - timedelta(seconds=NUKE_WINDOW)
    tracker = nuke_tracker[guild.id][user.id]
    tracker.append(now)
    nuke_tracker[guild.id][user.id] = [t for t in tracker if t > cutoff]
    count = len(nuke_tracker[guild.id][user.id])
    if count >= NUKE_THRESHOLD:
        nuke_tracker[guild.id][user.id] = []
        log_channel = guild.system_channel
        actions_taken = []
        try:
            member = guild.get_member(user.id)
            if member:
                bot_top_role = guild.me.top_role
                removable = [r for r in member.roles if r.name != "@everyone" and
                             if removable:
                                 await member.remove_roles(*removable, reason="[Anti-Nuke]")
                                 actions_taken.append("Rollen entfernt")
        try:
            await guild.ban(user, reason=f"[Anti-Nuke] {count}x {action_label}")
            actions_taken.append("Gebannt")
        except discord.Forbidden:
            actions_taken.append("Ban fehlgeschlagen")
        msg = f"Anti-Nuke: {user} - {count}x {action_label} - {'', '}.join(actions_taken)"
    except Exception:
        msg = f"Anti-Nuke: {user} - Fehler"
    if log_channel:
        await log_channel.send(msg)

@bot.event
async def on_member_ban(guild, user):
    executor = await get_audit_executor(guild, discord.AuditLogAction.ban)
    await check_nuke(guild, executor, "Ban")

@bot.event
async def on_guild_channel_delete(channel):
    executor = await get_audit_executor(channel.guild, discord.AuditLogAction.channel_delete)
    await check_nuke(channel.guild, executor, "Channel-Loeschung")

@bot.event
async def on_guild_role_delete(role):
    executor = await get_audit_executor(role.guild, discord.AuditLogAction.role_delete)

```

```

        await check_nuke(role.guild, executor, "Rollen-Loeschung")

@bot.event
async def on_member_remove(member):
    executor = await get_audit_executor(member.guild, discord.AuditLogAction.kick)
    if executor:
        await check_nuke(member.guild, executor, "Kick")

@bot.event
async def on_ready():
    await tree.sync()
    print(f"Bot ist online als {bot.user}")
    print("Slash Commands synchronisiert!")

# -----
# /help
# -----

@tree.command(name="help", description="Zeigt alle Befehle")
async def help_cmd(interaction: discord.Interaction):
    embed = liquid_glass_embed(
        "Bot Befehle",
        "Alle verfügbaren Slash-Commands auf einen Blick.",
        discord.Color.from_rgb(100, 180, 255)
    )
    embed.add_field(
        name="⚙️ Moderation",
        value="`/kick` `/teamkick` `/tempmute` `/unmute` `/teamwarn` \n`/warnings`",
        inline=False
    )
    embed.add_field(
        name="🔔 Alerts",
        value="`/setalerts` `/removealerts`",
        inline=False
    )
    embed.add_field(
        name="🇩🇪 Server",
        value="`/serverstatus` `/userinfo` `/status` `/ankündigung`",
        inline=False
    )
    await interaction.response.send_message(embed=embed)

# -----
# /hallo
# -----

@tree.command(name="hallo", description="Begrueßung")

```

```

async def hallo(interaction: discord.Interaction):
    embed = liquid_glass_embed("Hallo!", f"Hey {interaction.user.mention} 🙌")
    await interaction.response.send_message(embed=embed)

# -----
# /embed - Liquid Glass Embed senden (Admin)
# -----

@tree.command(name="embed", description="Sendet eine Nachricht als Liquid Glass E
@app_commands.describe(titel="Titel des Embeds", nachricht="Nachricht / Beschreib
async def embed_cmd(interaction: discord.Interaction, titel: str, nachricht: str)
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    embed = liquid_glass_embed(titel, nachricht)
    embed.set_author(
        name=interaction.user.display_name,
        icon_url=interaction.user.display_avatar.url
    )
    await interaction.response.send_message(embed=embed)

# -----
# /serverstatus (Admin)
# -----

@tree.command(name="serverstatus", description="Zeigt den aktuellen Server-Status
async def serverstatus(interaction: discord.Interaction):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    guild = interaction.guild

    total_members    = guild.member_count
    online_members    = sum(1 for m in guild.members if m.status != discord.Status.
    bots              = sum(1 for m in guild.members if m.bot)
    text_channels     = len(guild.text_channels)
    voice_channels    = len(guild.voice_channels)
    roles             = len(guild.roles) - 1 # @everyone raus
    boost_level       = guild.premium_tier
    boosts            = guild.premium_subscription_count
    created_at        = guild.created_at.strftime("%d.%m.%Y")

    embed = liquid_glass_embed(
        f"Server Status - {guild.name}",
        color=discord.Color.from_rgb(80, 210, 200)
    )
    if guild.icon:

```

```

        embed.set_thumbnail(url=guild.icon.url)

    embed.add_field(name="👥 Mitglieder", value=f"`{total_members}`", inline=True)
    embed.add_field(name="🟢 Online", value=f"`{online_members}`", inline=True)
    embed.add_field(name="🤖 Bots", value=f"`{bots}`", inline=True)
    embed.add_field(name="💬 Text-Channels", value=f"`{text_channels}`", inline=True)
    embed.add_field(name="🎤 Voice-Channels", value=f"`{voice_channels}`", inline=True)
    embed.add_field(name="👤 Rollen", value=f"`{roles}`", inline=True)
    embed.add_field(name="🌟 Boost Level", value=f"`{boost_level}`", inline=True)
    embed.add_field(name="🚀 Boosts", value=f"`{boosts}`", inline=True)
    embed.add_field(name="📅 Erstellt am", value=f"`{created_at}`", inline=True)

    await interaction.response.send_message(embed=embed)

# -----
# /kick (Admin)
# -----

@tree.command(name="kick", description="Kickt einen User vom Server")
@app_commands.describe(member="Der User", grund="Grund")
async def kick(interaction: discord.Interaction, member: discord.Member, grund: str):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=True)
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemer ist immun!", ephemeral=True)
        return
    try:
        await member.kick(reason=grund)
        embed = liquid_glass_embed(
            "Kick",
            f"**{member}** wurde vom Server gekickt.\n**Grund:** {grund}",
            discord.Color.from_rgb(255, 160, 100)
        )
        embed.add_field(name="Moderator", value=interaction.user.mention, inline=True)
        await interaction.response.send_message(embed=embed)
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.", ephemeral=True)

# -----
# /teamkick (Admin)
# -----

@tree.command(name="teamkick", description="Entfernt alle Rollen eines Users")
@app_commands.describe(member="Der User")
async def teamkick(interaction: discord.Interaction, member: discord.Member):
    if not interaction.user.guild_permissions.administrator:

```

```

        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuerer ist immun!", eph
        return
    roles = [r for r in member.roles if r.name != "@everyone"]
    if not roles:
        await interaction.response.send_message(f"**{member}** hat keine Rollen."
        return
    await member.remove_roles(*roles)
    embed = liquid_glass_embed(
        "Team-Kick",
        f"Alle Rollen von **{member}** wurden entfernt.",
        discord.Color.from_rgb(255, 140, 80)
    )
    await interaction.response.send_message(embed=embed)

# -----
# /tempmute (Admin)
# -----

@tree.command(name="tempmute", description="Schaltet einen User stumm")
@app_commands.describe(member="Der User", minuten="Dauer in Minuten", grund="Grund")
async def tempmute(interaction: discord.Interaction, member: discord.Member, minu
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuerer ist immun!", eph
        return
    until = datetime.now(timezone.utc) + timedelta(minutes=minuten)
    try:
        await member.timeout(until, reason=grund)
        embed = liquid_glass_embed(
            "Temp-Mute",
            f"**{member}** wurde für **{minuten} Minuten** stummgeschaltet.\n**Gr
            discord.Color.from_rgb(200, 100, 240)
        )
        await interaction.response.send_message(embed=embed)
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.", ephemer

# -----
# /unmute (Admin)
# -----

@tree.command(name="unmute", description="Hebt den Mute auf")

```

```

@app_commands.describe(member="Der User")
async def unmute(interaction: discord.Interaction, member: discord.Member):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemmer ist immun!", eph
        return
    try:
        await member.timeout(None)
        embed = liquid_glass_embed(
            "Unmute",
            f"Der Mute von **{member}** wurde aufgehoben.",
            discord.Color.from_rgb(100, 220, 150)
        )
        await interaction.response.send_message(embed=embed)
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.", ephemer

# -----
# /teamwarn - KEIN Ping (Admin)
# -----

@tree.command(name="teamwarn", description="Verwarnt einen User (kein Ping)")
@app_commands.describe(member="Der User", grund="Grund")
async def teamwarn(interaction: discord.Interaction, member: discord.Member, grun
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemmer ist immun!", eph
        return
    user_id = str(member.id)
    if user_id not in warnings_data:
        warnings_data[user_id] = []
    warnings_data[user_id].append({
        "reason": grund,
        "by": str(interaction.user),
        "at": datetime.now(timezone.utc).isoformat()
    })
    save_warnings(warnings_data)
    count = len(warnings_data[user_id])

# Kein mention - stattdessen nur Name#Discriminator
embed = liquid_glass_embed(
    "Verwarnung",
    f"**{member}** wurde verwarnt.\n**Grund:** {grund}\n**Verwarnung Nr.:** {

```



```

        discord.Color.from_rgb(255, 200, 60)
    )
    embed.add_field(name="Moderator", value=str(interaction.user), inline=True)
    await interaction.response.send_message(embed=embed, allowed_mentions=discord.

if count >= 3:
    try:
        await member.ban(reason=f"Auto-Ban nach {count} Verwarnungen")
        warnings_data[user_id] = []
        save_warnings(warnings_data)
        ban_embed = liquid_glass_embed(
            "Auto-Ban",
            f"**{member}** wurde nach {count} Verwarnungen automatisch gebann
            discord.Color.from_rgb(220, 60, 60)
        )
        await interaction.followup.send(embed=ban_embed, allowed_mentions=dis
    except discord.Forbidden:
        await interaction.followup.send("Fehlende Berechtigung zum Bannen.")

# -----
# /warnings (Admin)
# -----

@tree.command(name="warnings", description="Zeigt Verwarnungen eines Users")
@app_commands.describe(member="Der User")
async def warnings_cmd(interaction: discord.Interaction, member: discord.Member):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    user_warnings = warnings_data.get(str(member.id), [])
    if not user_warnings:
        await interaction.response.send_message(f"**{member}** hat keine Verwarnu
        return
    embed = liquid_glass_embed(
        f"Verwarnungen - {member}",
        f"Insgesamt **{len(user_warnings)}** Verwarnung(en).",
        discord.Color.from_rgb(255, 180, 50)
    )
    for i, w in enumerate(user_warnings, 1):
        embed.add_field(name=f"# {i} - {w['reason']}", value=f"von {w['by']}", inl
    await interaction.response.send_message(embed=embed, allowed_mentions=discord

# -----
# /allwarnings (Admin)
# -----

@tree.command(name="allwarnings", description="Alle aktiven Verwarnungen")

```

```

async def allwarnings(interaction: discord.Interaction):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    active = {uid: w for uid, w in warnings_data.items() if w}
    if not active:
        await interaction.response.send_message("Keine aktiven Verwarnungen.")
        return
    embed = liquid_glass_embed("Alle Verwarnungen", color=discord.Color.from_rgb(
    for uid, warns in sorted(active.items(), key=lambda x: len(x[1]), reverse=True)
        try:
            u = await bot.fetch_user(int(uid))
            name = str(u)
        except Exception:
            name = uid
        embed.add_field(name=f"{name} - {len(warns)}x", value=warns[-1]["reason"])
    await interaction.response.send_message(embed=embed)

# -----
# /clearwarnings (Admin)
# -----

@tree.command(name="clearwarnings", description="Loescht alle Verwarnungen eines
@app_commands.describe(member="Der User")
async def clearwarnings(interaction: discord.Interaction, member: discord.Member)
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    user_id = str(member.id)
    if not warnings_data.get(user_id):
        await interaction.response.send_message(f"**{member}** hat keine Verwarnu
        return
    count = len(warnings_data[user_id])
    warnings_data[user_id] = []
    save_warnings(warnings_data)
    embed = liquid_glass_embed(
        "Verwarnungen gelöscht",
        f"**{count}** Verwarnung(en) von **{member}** wurden gelöscht.",
        discord.Color.from_rgb(100, 220, 150)
    )
    await interaction.response.send_message(embed=embed, allowed_mentions=discord

# -----
# /bann (Admin)
# -----

@tree.command(name="bann", description="Bannt einen User")

```

```

@app_commands.describe(member="Der User", grund="Grund")
async def bann(interaction: discord.Interaction, member: discord.Member, grund: s
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemmer ist immun!", eph
        return
    try:
        await member.ban(reason=grund)
        embed = liquid_glass_embed(
            "Ban",
            f"**{member}** wurde gebannt.\n**Grund:** {grund}",
            discord.Color.from_rgb(220, 60, 60)
        )
        embed.add_field(name="Moderator", value=str(interaction.user), inline=True
        await interaction.response.send_message(embed=embed)
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.", ephemer

```

```

# -----
# /unban (Admin)
# -----

```

```

@tree.command(name="unban", description="Entbannt einen User per ID")
@app_commands.describe(user_id="Die User-ID")
async def unban(interaction: discord.Interaction, user_id: str):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    try:
        user = await bot.fetch_user(int(user_id))
        await interaction.guild.unban(user)
        embed = liquid_glass_embed(
            "Unban",
            f"**{user}** wurde entbannt.",
            discord.Color.from_rgb(100, 220, 150)
        )
        await interaction.response.send_message(embed=embed)
    except Exception:
        await interaction.response.send_message("Fehler beim Entbannen.", ephemer

```

```

# -----
# /setalerts & /removealerts
# -----

```

```

@tree.command(name="setalerts", description="DM-Alerts aktivieren")

```

```

async def setalerts(interaction: discord.Interaction):
    uid = str(interaction.user.id)
    if uid in config_data["alert_users"]:
        await interaction.response.send_message("Du hast bereits Alerts.", ephemeral=True)
        return
    config_data["alert_users"].append(uid)
    save_config(config_data)
    await interaction.response.send_message("✅ DM-Alerts aktiviert!", ephemeral=True)

@tree.command(name="removealerts", description="DM-Alerts deaktivieren")
async def removealerts(interaction: discord.Interaction):
    uid = str(interaction.user.id)
    if uid not in config_data["alert_users"]:
        await interaction.response.send_message("Keine aktiven Alerts.", ephemeral=True)
        return
    config_data["alert_users"].remove(uid)
    save_config(config_data)
    await interaction.response.send_message("🚫 DM-Alerts deaktiviert.", ephemeral=True)

# -----
# /clear (Admin) - alle oder User-Nachrichten löschen
# -----

@tree.command(name="clear", description="Löscht Nachrichten im Channel (optional:
@app_commands.describe(anzahl="Anzahl der Nachrichten (max. 100)", member="Nur Na
async def clear(interaction: discord.Interaction, anzahl: int = 100, member: disc
    if not interaction.user.guild_permissions.manage_messages:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=True)
        return
    await interaction.response.defer(ephemeral=True)
    if anzahl > 100:
        anzahl = 100
    if member:
        deleted = []
        async for msg in interaction.channel.history(limit=200):
            if msg.author == member:
                deleted.append(msg)
            if len(deleted) >= anzahl:
                break
        for msg in deleted:
            try:
                await msg.delete()
                await asyncio.sleep(0.5)
            except Exception:
                pass
        count = len(deleted)
        embed = liquid_glass_embed(

```

```

        "Clear",
        f"***{count}** Nachricht(en) von **{member}** wurden gelöscht.",
        discord.Color.from_rgb(255, 100, 100)
    )
else:
    deleted = await interaction.channel.purge(limit=anzahl)
    count = len(deleted)
    embed = liquid_glass_embed(
        "Clear",
        f"***{count}** Nachricht(en) wurden gelöscht.",
        discord.Color.from_rgb(255, 100, 100)
    )
    await interaction.followup.send(embed=embed, ephemeral=True)

# -----
# /userinfo
# -----

@tree.command(name="userinfo", description="Zeigt Infos über einen User")
@app_commands.describe(member="Der User (leer = du selbst)")
async def userinfo(interaction: discord.Interaction, member: discord.Member = None):
    member = member or interaction.user
    roles = [r.mention for r in reversed(member.roles) if r.name != "@everyone"]
    joined = member.joined_at.strftime("%d.%m.%Y %H:%M") if member.joined_at else None
    created = member.created_at.strftime("%d.%m.%Y %H:%M")
    embed = liquid_glass_embed(
        f"Userinfo - {member}",
        color=discord.Color.from_rgb(130, 200, 240)
    )
    embed.set_thumbnail(url=member.display_avatar.url)
    embed.add_field(name="🆔 ID", value=f"`{member.id}`", inline=True)
    embed.add_field(name="👤 Name", value=str(member), inline=True)
    embed.add_field(name="🤖 Bot", value="Ja" if member.bot else "Nein", inline=True)
    embed.add_field(name="📅 Account erstellt", value=f"`{created}`", inline=True)
    embed.add_field(name="📅 Server beigetreten", value=f"`{joined}`", inline=True)
    embed.add_field(name="👥 Rollen", value=", ".join(roles) if roles else "", inline=True)
    await interaction.response.send_message(embed=embed)

# -----
# /status - Bot-Status setzen (Owner only)
# -----

@tree.command(name="status", description="Setzt den Bot-Status (nur Owner)")
@app_commands.describe(text="Status-Text", typ="Typ: playing / watching / listen")
@app_commands.choices(typ=[
    app_commands.Choice(name="playing", value="playing"),
    app_commands.Choice(name="watching", value="watching"),
])

```

```

        app_commands.Choice(name="listening", value="listening"),
        app_commands.Choice(name="streaming", value="streaming"),
    ])
    async def status_cmd(interaction: discord.Interaction, text: str, typ: str = "pla
        if not is_owner(interaction.user):
            await interaction.response.send_message("Nur der Owner kann den Status än
            return
        activity_map = {
            "playing": discord.Game(name=text),
            "watching": discord.Activity(type=discord.ActivityType.watching, name=t
            "listening": discord.Activity(type=discord.ActivityType.listening, name=t
            "streaming": discord.Streaming(name=text, url="https://twitch.tv/placehol
        }
        await bot.change_presence(activity=activity_map.get(typ, discord.Game(name=te
        embed = liquid_glass_embed("Status geändert", f"Status: **{typ}** `{text}`",
        await interaction.response.send_message(embed=embed, ephemeral=True)

# -----
# /ankündigung (Admin)
# -----

@tree.command(name="ankündigung", description="Schickt eine Ankündigung in einen
@app_commands.describe(kanal="Ziel-Kanal", titel="Titel der Ankündigung", nachric
    async def ankuendung(interaction: discord.Interaction, kanal: discord.TextChann
        if not interaction.user.guild_permissions.administrator:
            await interaction.response.send_message("Keine Berechtigung!", ephemeral=
            return
        embed = liquid_glass_embed(titel, nachricht, discord.Color.from_rgb(255, 200,
        embed.set_author(name=interaction.user.display_name, icon_url=interaction.use
        embed.set_footer(text="🔊 Ankündigung")
        await kanal.send(embed=embed)
        await interaction.response.send_message(f"✅ Ankündigung wurde in {kanal.ment

# -----
# Anti-Link System
# -----

import re
LINK_PATTERN = re.compile(r"https?:\/\/|discord\.gg\/|www\.", re.IGNORECASE)

def get_antilink(guild_id: int) -> dict:
    cfg = load_config()
    return cfg.get("antilink", {}).get(str(guild_id), {
        "enabled": False,
        "timeout_minutes": 5,
        "delete_message": True,
        "ignored_users": [],

```

```

        "ignored_roles": []
    })

def save_antilink(guild_id: int, data: dict):
    cfg = load_config()
    if "antilink" not in cfg:
        cfg["antilink"] = {}
    cfg["antilink"][str(guild_id)] = data
    save_config(cfg)

@bot.event
async def on_message(message: discord.Message):
    if message.author.bot or not message.guild:
        await bot.process_commands(message)
        return

    settings = get_antilink(message.guild.id)

    if settings.get("enabled"):
        # Check ignored users
        if message.author.id not in settings.get("ignored_users", []):
            # Check ignored roles
            member_role_ids = [r.id for r in message.author.roles]
            ignored_roles = settings.get("ignored_roles", [])
            if not any(r in member_role_ids for r in ignored_roles):
                if LINK_PATTERN.search(message.content):
                    # Delete message
                    if settings.get("delete_message", True):
                        try:
                            await message.delete()
                        except Exception:
                            pass
                    # Timeout user
                    minutes = settings.get("timeout_minutes", 5)
                    if minutes > 0:
                        try:
                            until = datetime.now(timezone.utc) + timedelta(minutes)
                            await message.author.timeout(until, reason="Anti-Link")
                        except Exception:
                            pass
                    # Warn in channel
                    embed = liquid_glass_embed(
                        "🔒 Anti-Link",
                        f"{message.author.mention} Links sind auf diesem Server n
                        discord.Color.from_rgb(255, 80, 80)
                    )
                    try:

```

```

        warn_msg = await message.channel.send(embed=embed)
        await asyncio.sleep(5)
        await warn_msg.delete()
    except Exception:
        pass

    await bot.process_commands(message)

# -----
# /antilink setup
# -----

@tree.command(name="antilink", description="Anti-Link System einstellen")
@app_commands.describe(
    aktiv="Anti-Link aktivieren oder deaktivieren",
    timeout_minuten="Wie lange wird der User getimeoutet (0 = kein Timeout)",
    nachricht_loeschen="Soll die Nachricht mit dem Link gelöscht werden?",
)
async def antilink(
    interaction: discord.Interaction,
    aktiv: bool,
    timeout_minuten: int = 5,
    nachricht_loeschen: bool = True,
):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=True)
        return

    settings = get_antilink(interaction.guild.id)
    settings["enabled"] = aktiv
    settings["timeout_minutes"] = timeout_minuten
    settings["delete_message"] = nachricht_loeschen
    save_antilink(interaction.guild.id, settings)
    status = "✅ Aktiviert" if aktiv else "❌ Deaktiviert"
    embed = liquid_glass_embed(
        "Anti-Link Einstellungen",
        f"***Status:** {status}\n***Timeout:** {timeout_minuten} Minuten\n***Nachricht löschen:** {"Ja" if nachricht_loeschen else "Nein"}",
        discord.Color.from_rgb(100, 220, 150) if aktiv else discord.Color.from_rgb(255, 0, 0)
    )
    await interaction.response.send_message(embed=embed)

@tree.command(name="antilink-ignore-user", description="User vom Anti-Link System ignorieren")
@app_commands.describe(member="Der User", aktion="Hinzufügen oder entfernen")
@app_commands.choices(aktion=[
    app_commands.Choice(name="hinzufügen", value="add"),
    app_commands.Choice(name="entfernen", value="remove"),
])
async def antilink_ignore_user(interaction: discord.Interaction, member: discord.Member, aktion: str):

```



```

if not interaction.user.guild_permissions.administrator:
    await interaction.response.send_message("Keine Berechtigung!", ephemeral=
    return
settings = get_antilink(interaction.guild.id)
ignored = settings.get("ignored_users", [])
if aktion == "add":
    if member.id not in ignored:
        ignored.append(member.id)
        msg = f"***{member}** wird jetzt ignoriert."
    else:
        ignored = [u for u in ignored if u != member.id]
        msg = f"***{member}** wird nicht mehr ignoriert."
    settings["ignored_users"] = ignored
    save_antilink(interaction.guild.id, settings)
    embed = liquid_glass_embed("Anti-Link • User", msg, discord.Color.from_rgb(13
    await interaction.response.send_message(embed=embed)

@tree.command(name="antilink-ignore-rolle", description="Rolle vom Anti-Link Syst
@app_commands.describe(rolle="Die Rolle", aktion="Hinzufügen oder entfernen")
@app_commands.choices(aktion=[
    app_commands.Choice(name="hinzufügen", value="add"),
    app_commands.Choice(name="entfernen", value="remove"),
])

async def antilink_ignore_rolle(interaction: discord.Interaction, rolle: discord.
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    settings = get_antilink(interaction.guild.id)
    ignored = settings.get("ignored_roles", [])
    if aktion == "add":
        if rolle.id not in ignored:
            ignored.append(rolle.id)
            msg = f"***{rolle.name}** wird jetzt ignoriert."
        else:
            ignored = [r for r in ignored if r != rolle.id]
            msg = f"***{rolle.name}** wird nicht mehr ignoriert."
        settings["ignored_roles"] = ignored
        save_antilink(interaction.guild.id, settings)
        embed = liquid_glass_embed("Anti-Link • Rolle", msg, discord.Color.from_rgb(1
        await interaction.response.send_message(embed=embed)

@tree.command(name="antilink-status", description="Zeigt die aktuellen Anti-Link
async def antilink_status(interaction: discord.Interaction):
    if not interaction.user.guild_permissions.administrator:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    settings = get_antilink(interaction.guild.id)

```

```

enabled = settings.get("enabled", False)
timeout = settings.get("timeout_minutes", 5)
delete = settings.get("delete_message", True)
ignored_users = settings.get("ignored_users", [])
ignored_roles = settings.get("ignored_roles", [])
users_str = ", ".join(f"<@{u}>" for u in ignored_users) if ignored_users else ""
roles_str = ", ".join(f"<@&{r}>" for r in ignored_roles) if ignored_roles else ""
embed = liquid_glass_embed(
    "Anti-Link Status",
    f"**Status:** {'✅ Aktiv' if enabled else '❌ Inaktiv'}\n**Timeout:** {ti
    discord.Color.from_rgb(130, 200, 240)
)
await interaction.response.send_message(embed=embed, ephemeral=True)

# -----
# Start
# -----

token = os.environ.get("DISCORD_TOKEN")
if not token:
    raise RuntimeError("DISCORD_TOKEN nicht gesetzt!")

bot.run(token)

```